



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика, искусственный интеллект и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ

по лабораторной работе № 1

по курсу «Методы вычислений»

на тему: «Венгерский метод решения задачи о назначениях»

Студент ИУ7-13М
(Группа)

(Подпись, дата)

Орду М. А.
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Власов П. А.
(И. О. Фамилия)

2025 г.

1 Теоретический раздел

1.1 Содержание работы

Цель работы: изучение венгерского метода решения задачи о назначениях.

1. реализовать венгерский метод решения задачи о назначениях в виде программы на ЭВМ;
2. провести решение задачи с матрицей стоимостей, заданной в индивидуальном варианте, рассмотрев два случая:
 - (а) задача о назначениях является задачей минимизации,
 - (б) задача о назначениях является задачей максимизации.

1.2 Постановка задачи

В распоряжении работодателя имеется n работ и n исполнителей. Стоимость выполнения i -ой работы j -ым исполнителем составляет $c_{ij} \geq 0$ единиц. Требуется распределить работу между исполнителями так, чтобы:

1. каждый исполнитель выполнял ровно одну работу;
2. общая стоимость выполнения всех работ была минимальной.

1.2.1 Математическая формулировка

Обозначим $C = (c_{ij})_{i,j=\overline{1;n}}$, где C – матрица стоимостей. Введем управляемые переменные:

$$x_{ij} = \begin{cases} 1, & \text{если } i\text{-ую работу выполняет } j\text{-ый работник} \\ 0, & \text{иначе} \end{cases}, \quad (1.1)$$

где $i, j = \overline{1, n}$

Обозначим $X = (x_{ij}), i, j = \overline{1, n}$, где X – матрица назначений.

Тогда общая стоимость выполнения работ:

$$f = \sum_{i=1}^n \sum_{j=1}^n c_{ij} \cdot x_{ij} \quad (1.2)$$

Условие того, что j -й работник выполняет ровно одну работу:

$$\sum_{i=1}^n x_{ij} = 1, \quad (1.3)$$

где $j = \overline{1; n}$

Условие того, что i -ю работу выполняет один работник:

$$\sum_{j=1}^n x_{ij} = 1, \quad (1.4)$$

где $i = \overline{1; n}$

Тогда математическая постановка задачи о назначениях имеет следующий вид:

$$x_{ij}(x) = \begin{cases} f = \sum_{i=1}^n \sum_{j=1}^n c_{ij} \cdot x_{ij} \rightarrow \min \\ \sum_{i=1}^n x_{ij} = 1, & j = \overline{1; n} \\ \sum_{j=1}^n x_{ij} = 1, & i = \overline{1; n} \\ x_{i,j} \in 0, 1, & i, j = \overline{1; n} \end{cases}, \quad (1.5)$$

где f – целевая функция, c_{ij} – элементы матрицы стоимостей C , x_{ij} – элементы матрицы назначений X .

1.2.2 Исходные данные индивидуального варианта

$$C = \begin{bmatrix} 3 & 5 & 2 & 4 & 8 \\ 10 & 10 & 4 & 3 & 6 \\ 5 & 6 & 9 & 8 & 3 \\ 6 & 2 & 5 & 8 & 4 \\ 5 & 4 & 8 & 9 & 3 \end{bmatrix}$$

1.3 Описание венгерского метода

Венгерский метод решения задачи о назначениях применяется для решения задачи минимизации, то есть для случая, когда матрица стоимостей C содержит в себе стоимости работ. Также матрица стоимостей C может использоваться для решения задачи максимизации, в этом случае ее элементы отражают прибыль работодателя от назначения j -го работника на i -ю работу.

Решение задачи максимизации отличается от решения задачи минимизации тем, что в данном случае в начале алгоритма добавляется шаг, на котором необходимо найти максимальный элемент столбца, вычесть его из каждого элемента матрицы и домножить матрицу на -1 .

Если решается задача минимизации, алгоритм начинается со следующих шагов:

1. в каждом столбце матрицы стоимостей выбирается наименьший элемент и вычитается из всех элементов этого столбца;
2. в каждой строке полученной матрицы выбирается наименьший элемент и вычитается из всех элементов этой строки.

В результате получается матрица, в каждой строке и столбце которой присутствует хотя бы один нуль.

Системой независимых нулей (СНН) будем называть такой набор нулей матрицы стоимостей, в котором никакие два элемента не стоят ни в общей строке, ни в общем столбце.

Для построения начальной СНН необходимо просмотреть столбцы текущей матрицы с размерностью $n \times n$ и первый найденный в столбце 0, в одной строке с которым нет элемента 0^* , отметить как 0^* .

Если полученная СНН содержит n элементов, значит, найдено оптимальное решение. Иначе СНН необходимо улучшить.

Для улучшения СНН необходимо:

1. отметить $+$ столбцы, в которых стоят 0^* (данные столбцы и все их элементы будем называть выделенными);
2. найти 0 среди невыделенных элементов и отметить его $0'$;

3. если в одной строке с $0'$ стоит 0^* , снять выделение со столбца с найденным 0^* и отметить строку с текущим $0'$, затем повторить поиск;
4. если в строке с выделенным $0'$ нет элемента 0^* , то строится L-цепочка:

$$0' \rightarrow \text{по столбцу} \rightarrow 0^* \rightarrow \text{по строке} \rightarrow 0' \rightarrow \dots \rightarrow 0'$$

В пределах цепочки заменить элементы: $0^* \rightarrow 0, 0' \rightarrow 0^*$

Если среди невыделенных элементов не найдено нулей, то необходимо:

1. среди всех невыделенных элементов выбрать наименьший элемент $h > 0$;
2. вычесть h из элементов в невыделенных столбцах;
3. прибавить h к элементам в выделенных строках.

В результате преобразований получим эквивалентную матрицу, среди невыделенных элементов которой есть 0, после этого приступить к улучшению СНН.

Схема венгерского метода решения задачи о назначениях представлена на рисунке 1.1.

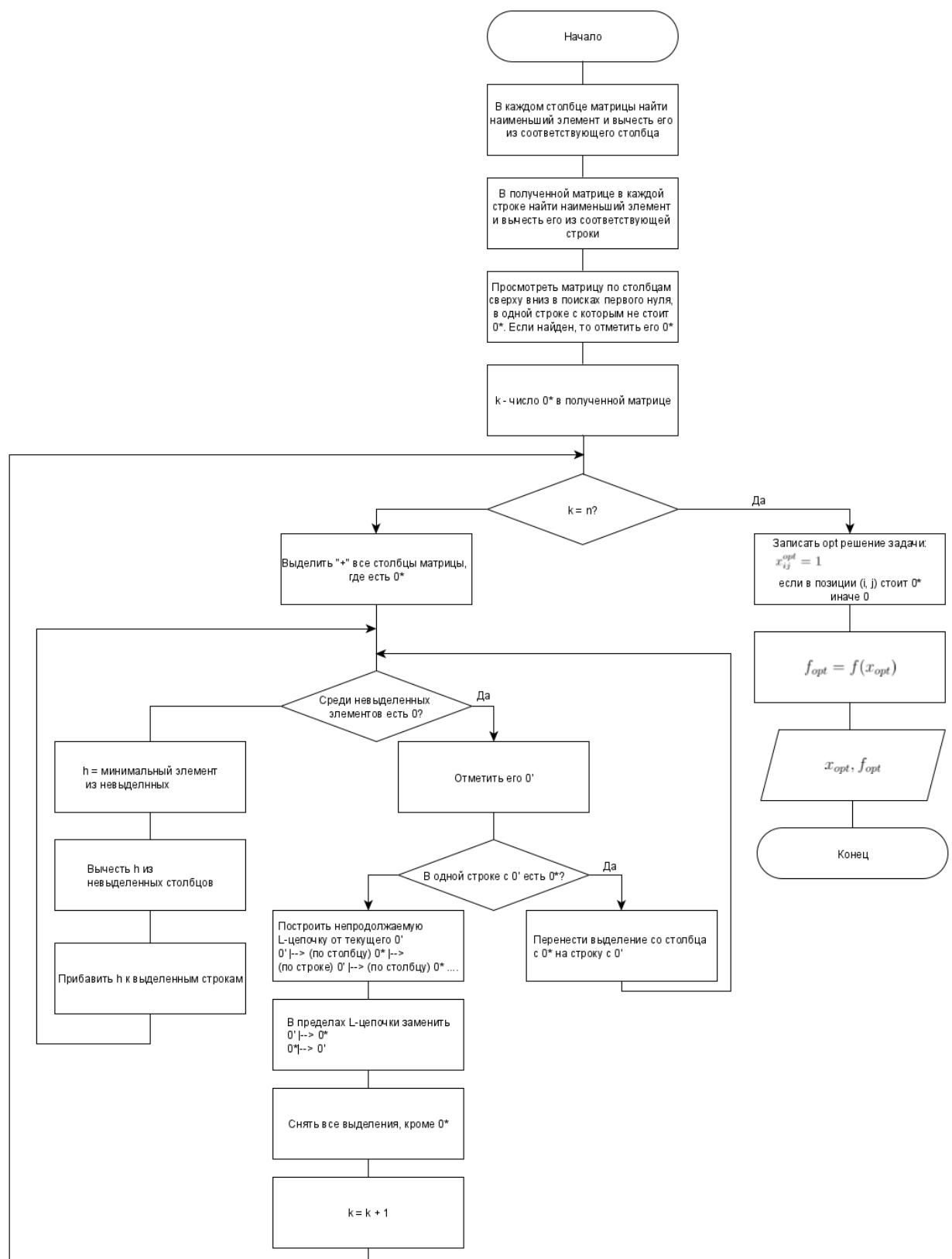


Рисунок 1.1 – Функциональная схема венгерского метода решения задачи о назначениях

2 Практический раздел

2.1 Программная реализация

В листинге 2.1 приведен текст программы.

Листинг 2.1 – Исходный код программы

```
1 % Лабораторная работа 1
2 % Решение задачи о назначениях венгерским методом
3 function mv_lab1()
4
5     % Параметры для отладки и выбора типа задачи
6     debug = true;           % включить вывод шагов
7     maximization = true;    % задача на максимум
8
9     % Исходная матрица стоимостей
10    C = [3, 5, 2, 4, 8;
11         10, 10, 4, 3, 6;
12         5, 6, 9, 8, 3;
13         6, 2, 5, 8, 4;
14         5, 4, 8, 9, 3];
15
16    [nRows, nCols] = size(C);
17    A = C;
18
19    % Если задача на максимум:
20    % находим максимальный элемент столбца, вычитаем его
21    % из каждого элемента матрицы и домножаем матрицу на 1
22    if maximization == true
23        maxVal = 0;
24        for i = 1:nRows
25            for j = 1:nCols
26                if C(i,j) > maxVal
27                    maxVal = C(i,j); % Находим максимальный
                                     элемент матрицы
28                end
29            end
30        end
31        for i = 1:nRows
32            for j = 1:nCols
33                A(i,j) = C(i,j) - maxVal; % Вычитаем
                                     максимальный элемент
```

```

34         % из всех элементов матрицы
35     end
36 end
37 A = -A; % Умножаем матрицу на -1
38 if debug
39     disp("Преобразование матрицы для задачи на
40         максимум");
41     disp(A);
42 end
43
44 % Вычитание минимальных элементов
45 % По столбцам
46 minCol = min(A, [], 1); % Находим минимальный элемент
47     каждого столбца
48 A = A - minCol; % Вычитаем
49 if debug
50     disp("Вычитание минимальных элементов по столбцам:");
51     disp(A);
52 end
53
54 % По строкам
55 minRow = min(A, [], 2); % Находим минимальный элемент
56     каждой строки
57 A = A - minRow; % Вычитаем
58 if debug
59     disp("Вычитание минимальных элементов по строкам:");
60     disp(A);
61 end
62
63 % Строим начальную СНН
64 stars = initStars(A);
65 N = size(A, 1);
66 quotes = zeros(N);
67
68 if debug
69     disp("Начальные 0*");
70     showDebug(A, stars, quotes, [], zeros(N));
71 end
72
73 % Считаем количество нулей в СНН

```

```

72     k = nnz(stars); % Считаем ненулевые элементны в м-це stars
73     if debug
74         disp("k = ");
75         disp(k);
76     end
77
78     % Если k < n - нужно улучшать
79     if k < nCols
80         if debug, disp("k нужно улучшить."); end
81
82         iter = 1;
83         while k < nRows
84             if debug
85                 fprintf("Итерация %d\n", iter);
86             end
87
88             markedCols = findMarkedCols(stars);
89             markedRows = zeros(size(A,2));
90
91             while true
92                 % Находим 0'
93                 [found, col, row] = findFreeZero(A,
94                     markedCols, markedRows);
95                 if found
96                     % Если невыделенный нуль найден
97                     quotes(row,col) = 1;
98                     if debug
99                         disp("После нахождения 0'");
100                         showDebug(A, stars, quotes,
101                             markedCols, markedRows);
102                     end
103                     % Проверяем есть ли с ним в одной строке 0*
104                     [hasStar, col2] = hasStarInRow(row, stars);
105                     if hasStar
106                         % Если есть - снимаем выделение с
107                         % этого столбца
108                         % и выделяем строку с 0'
109                         markedRows(row) = 1;
110                         markedCols(col2) = 0;
111                         if debug

```

```

109         disp("Изменили выделенные строки и
110             столбцы");
111     showDebug(A, stars, quotes,
112             markedCols, markedRows);
113     end
114 else
115     % Если нет - строим непродолжаемую
116     % L-цепочку
117     stars = buildLchain(stars, quotes,
118         row, col);
119     k = k + 1;
120     markedCols = findMarkedCols(stars);
121     markedRows = zeros(size(A,2));
122     if debug
123         disp("Новые 0* после построения
124             L-цепочки:");
125         disp(stars);
126     end
127     break;
128 end
129 else
130     % Если невыделенного нуля нет
131     % Находим наим эл-т h среди невыд эл-тов
132     % Вычитаем из невыд столбцов h
133     % Добавляем к выд строкам h
134     A = recalMatrix(A, markedRows,
135         markedCols);
136     if debug
137         disp("Пересчитали матрицу h");
138         showDebug(A, stars, quotes,
139             markedCols, markedRows);
140     end
141     end
142     iter = iter + 1;
143 end
144
145 % Оптимальное решение
146 X_opt = zeros(nCols);
147 for i = 1:nRows

```

```

143         for j = 1:nCols
144             if stars(i,j)
145                 X_opt(i,j) = 1;
146             end
147         end
148     end
149     disp("X_opt = ");
150     disp(X_opt);
151
152     f_opt = 0;
153     for i = 1:nRows
154         for j = 1:nCols
155             if stars(i,j)
156                 f_opt = f_opt + C(i,j);
157             end
158         end
159     end
160     disp("f_opt = ");
161     disp(f_opt);
162 end
163
164
165
166 % Построение начальной СНН
167 function stars = initStars(A)
168     N = size(A,1);
169     stars = zeros(N);
170     for j = 1:N
171         for i = 1:N
172             if A(i,j) == 0 && max(stars(:,j))==0 &&
173                 max(stars(i,:))==0
174                 stars(i,j) = 1;
175                 break;
176             end
177         end
178     end
179
180 % Поиск отмеченных столбцов
181 function cols = findMarkedCols(stars)
182     cols = sum(stars);

```

```

183 end
184
185 % Поиск неотмеченного нуля
186 function [found, col, row] = findFreeZero(A, markedCols,
    markedRows)
187     N = size(A,1);
188     found = false; col = 0; row = 0;
189     for j = 1:N
190         for i = 1:N
191             if A(i,j) == 0 && ~markedCols(j) && ~markedRows(i)
192                 found = true;
193                 col = j; row = i;
194                 return;
195             end
196         end
197     end
198 end
199
200 % Проверка наличия 0* в строке
201 function [hasStar, col] = hasStarInRow(row, stars)
202     hasStar = false; col = 0;
203     for j = 1:size(stars,2)
204         if stars(row,j) == 1
205             hasStar = true;
206             col = j;
207             return;
208         end
209     end
210 end
211
212 % Проверка наличия 0* в столбце
213 function [hasStar, row] = hasStarInCol(col, stars)
214     [hasStar, row] = hasStarInRow(col, stars');
215 end
216
217 % Пересчет матрицы через минимальный элемент h
218 function A = recalMatrix(A, markedRows, markedCols)
219     N = size(A,1);
220     h = inf;
221     for j = 1:N
222         for i = 1:N

```

```

223         if ~markedCols(j) && ~markedRows(i)
224             if A(i,j) < h
225                 h = A(i,j);
226             end
227         end
228     end
229 end
230
231 for j = 1:N
232     for i = 1:N
233         if ~markedCols(j) && ~markedRows(i)
234             A(i,j) = A(i,j) - h;
235         elseif markedCols(j) && markedRows(i)
236             A(i,j) = A(i,j) + h;
237         end
238     end
239 end
240 end
241
242 % Построение L-цепочки
243 function newStars = buildLchain(stars, quotes, row, col)
244     newStars = stars;
245     curRow = row; curCol = col;
246     flag = true;
247     while flag
248         newStars(curRow, curCol) = 1;
249         [flag, curRow] = hasStarInCol(curCol, stars);
250         if flag
251             newStars(curRow, curCol) = 0;
252             [flag, curCol] = hasStarInRow(curRow, quotes);
253         end
254     end
255 end
256
257 % Отладочный вывод
258 function showDebug(A, stars, quotes, markedCols, markedRows)
259     for i = 1:size(A,1)
260         for j = 1:size(A,2)
261             if stars(i,j)
262                 fprintf("%d* \t", A(i,j));
263             elseif quotes(i,j)

```

```

264         fprintf("%d' \t", A(i,j));
265     else
266         fprintf("%d \t", A(i,j));
267     end
268 end
269 if i <= length(markedRows) && markedRows(i)
270     fprintf(" +\n");
271 else
272     fprintf("\n");
273 end
274 end
275 for j = 1:length(markedCols)
276     if markedCols(j)
277         fprintf("+ \t");
278     else
279         fprintf(" \t");
280     end
281 end
282 fprintf("\n");
283 end

```

2.2 Результаты решений

2.2.1 Задача минимизации

Листинг 2.2 – Вывод программы при задаче минимизации

```

1
2  Вычитание минимальных элементов по столбцам:
3      0      3      0      1      5
4      7      8      2      0      3
5      2      4      7      5      0
6      3      0      3      5      1
7      2      2      6      6      0
8
9  Вычитание минимальных элементов по строкам:
10     0      3      0      1      5
11     7      8      2      0      3
12     2      4      7      5      0
13     3      0      3      5      1
14     2      2      6      6      0
15

```

```

16 Начальные 0*:
17 0* 3 0 1 5
18 7 8 2 0* 3
19 2 4 7 5 0*
20 3 0* 3 5 1
21 2 2 6 6 0
22
23 k =
24 4
25
26 k нужно улучшить.
27 Итерация 1
28 После нахождения 0'
29 0* 3 0' 1 5
30 7 8 2 0* 3
31 2 4 7 5 0*
32 3 0* 3 5 1
33 2 2 6 6 0
34 + + + +
35 Изменили выделенные строки и столбцы
36 0* 3 0' 1 5 +
37 7 8 2 0* 3
38 2 4 7 5 0*
39 3 0* 3 5 1
40 2 2 6 6 0
41 + + +
42 Пересчитали матрицу h
43 0* 5 0' 3 7 +
44 5 8 0 0* 3
45 0 4 5 5 0*
46 1 0* 1 5 1
47 0 2 4 6 0
48 + + +
49 После нахождения 0'
50 0* 5 0' 3 7 +
51 5 8 0 0* 3
52 0' 4 5 5 0*
53 1 0* 1 5 1
54 0 2 4 6 0
55 + + +
56 Изменили выделенные строки и столбцы

```

```

57 0* 5 0' 3 7 +
58 5 8 0 0* 3
59 0' 4 5 5 0* +
60 1 0* 1 5 1
61 0 2 4 6 0
62 + +
63 После нахождения 0'
64 0* 5 0' 3 7 +
65 5 8 0 0* 3
66 0' 4 5 5 0* +
67 1 0* 1 5 1
68 0' 2 4 6 0
69 + +
70 Новые 0* после построения L-цепочки:
71 0 0 1 0 0
72 0 0 0 1 0
73 0 0 0 0 1
74 0 1 0 0 0
75 1 0 0 0 0
76
77 X_opt =
78 0 0 1 0 0
79 0 0 0 1 0
80 0 0 0 0 1
81 0 1 0 0 0
82 1 0 0 0 0
83
84 f_opt =
85 15

```

2.2.2 Задача максимизации

Листинг 2.3 – Вывод программы при задаче максимизации

```

1 Преобразование матрицы для задачи на максимум
2 7 5 8 6 2
3 0 0 6 7 4
4 5 4 1 2 7
5 4 8 5 2 6
6 5 6 2 1 7
7
8 Вычитание минимальных элементов по столбцам:
9 7 5 7 5 0

```

10	0	0	5	6	2	
11	5	4	0	1	5	
12	4	8	4	1	4	
13	5	6	1	0	5	
14						
15	Вычитание минимальных элементов по строкам:					
16	7	5	7	5	0	
17	0	0	5	6	2	
18	5	4	0	1	5	
19	3	7	3	0	3	
20	5	6	1	0	5	
21						
22	Начальные 0*:					
23	7	5	7	5	0*	
24	0*	0	5	6	2	
25	5	4	0*	1	5	
26	3	7	3	0*	3	
27	5	6	1	0	5	
28						
29	k =					
30	4					
31						
32	k нужно улучшить.					
33	Итерация 1					
34	После нахождения 0'					
35	7	5	7	5	0*	
36	0*	0'	5	6	2	
37	5	4	0*	1	5	
38	3	7	3	0*	3	
39	5	6	1	0	5	
40	+		+	+	+	
41	Изменили выделенные строки и столбцы					
42	7	5	7	5	0*	
43	0*	0'	5	6	2	+
44	5	4	0*	1	5	
45	3	7	3	0*	3	
46	5	6	1	0	5	
47			+	+	+	
48	Пересчитали матрицу h					
49	4	2	7	5	0*	
50	0*	0'	8	9	5	+

```

51 2   1   0*  1   5
52 0   4   3   0*  3
53 2   3   1   0   5
54      +   +   +
55 После нахождения 0'
56 4   2   7   5   0*
57 0*  0'  8   9   5   +
58 2   1   0*  1   5
59 0'  4   3   0*  3
60 2   3   1   0   5
61      +   +   +
62 Изменили выделенные строки и столбцы
63 4   2   7   5   0*
64 0*  0'  8   9   5   +
65 2   1   0*  1   5
66 0'  4   3   0*  3   +
67 2   3   1   0   5
68      +           +
69 После нахождения 0'
70 4   2   7   5   0*
71 0*  0'  8   9   5   +
72 2   1   0*  1   5
73 0'  4   3   0*  3   +
74 2   3   1   0'  5
75      +           +
76 Новые 0* после построения L-цепочки:
77      0       0       0       0       1
78      0       1       0       0       0
79      0       0       1       0       0
80      1       0       0       0       0
81      0       0       0       1       0
82
83 X_opt =
84      0       0       0       0       1
85      0       1       0       0       0
86      0       0       1       0       0
87      1       0       0       0       0
88      0       0       0       1       0
89
90 f_opt =
91      42

```

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы был реализован венгерский метод решения задачи о назначениях для задач минимизации и максимизации. В ходе выполнения лабораторной работы была написана программа на языке MatLab, позволяющая применять венгерский метод решения задачи о назначениях